AUTOENCODERS

AND

VARIATIONAL AUTOENCODERS

From First Principles to Practice

A Deep Learning Textbook

Generative Modeling · Latent Variable Models · Probabilistic Deep Learning

Prepared for: Sunkeerth

Engineering — Robotics, AI & Autonomous Systems

Table of Contents

Table of Contents 2
Preface 6
Mathematical Prerequisites 7
From Calculus 7
From Linear Algebra 7
From Probability 7
PART I — Foundations: Rebuilding the Autoencoder 8
Chapter 1: What Problem Are We Actually Solving? 8
1.1 Roadmap of this Book 8
Chapter 2: The Plain Autoencoder 10
2.1 Definition 10
2.2 The Loss Function 10
2.3 A Minimal PyTorch Implementation 11
Chapter 3: Varieties of Autoencoders 13
3.1 Undercomplete vs. Overcomplete Autoencoders 13
3.2 Denoising Autoencoders (DAE) 13
3.3 Sparse Autoencoders 13
3.4 Contractive Autoencoders (CAE) 13
3.5 Summary Table 14
Chapter 4: Why Plain Autoencoders Cannot Generate 15
4.1 The Tempting (But Wrong) Idea 15
4.2 The Real Problem: An Unstructured Latent Space 15
4.3 What We Actually Need 16
PART II — The Probability Toolkit 17
Chapter 5: Random Variables and Distributions 18
5.1 What a Distribution Really Represents 18
5.2 Sampling 18
Chapter 6: KL Divergence 19
6.1 Measuring the Distance Between Two Distributions 19
6.2 Two Important Warnings 19
6.3 The One Formula You Actually Need 19
Chapter 7: Bayes' Rule and Latent Variable Models 21
7.1 Bayes' Rule Refresher 21
7.2 Latent Variable Models 21

7.3 The Problem: p(x) is Intractable 21
PART III — The Variational Autoencoder 23
Chapter 8: Setting Up the Variational Problem 24
8.1 Restating the Goal 24
8.2 Introducing an Approximate Posterior 24
Chapter 9: Deriving the ELBO, Step by Step 25
9.1 Starting Point 25
9.2 Bringing in Bayes' Rule 25
9.3 Why This is Exactly What We Need 25
9.4 Splitting the ELBO into Two Practical Terms 26
Chapter 10: The Reparameterization Trick 28
10.1 The Problem: Sampling is Not Differentiable 28
10.2 The Fix: Move the Randomness Outside 28
Chapter 11: Full VAE Architecture and Implementation 30
11.1 The Complete Picture 30
11.2 A Note on $\log\sigma^2$ 30
11.3 Full PyTorch Implementation 30
11.4 Generating New Samples After Training 31
Chapter 12: Training Dynamics and Common Failure Modes 33
12.1 What Healthy Training Looks Like 33
12.2 The Reconstruction vs. Regularization Trade-off 33
12.3 Posterior Collapse 33
12.4 Common Fixes 34
Chapter 13: A Complete Worked Numerical Example 35
13.1 Setup 35
13.2 Sampling z via the Reparameterization Trick 35
13.3 Computing the Reconstruction Loss 35
13.4 Computing the KL Term 36
13.5 The Final Loss 36
Chapter 14: Convolutional VAEs for Real Image Data 37
14.1 Why Flatten-and-Linear Doesn't Scale 37
14.2 A Convolutional Encoder/Decoder Sketch 37
14.3 Practical Tips for Image VAEs 38
PART IV — The VAE Family and Its Neighbors 39
Chapter 15: Conditional VAE (CVAE) 40
13.1 The Limitation of a Plain VAE 40
13.2 The Fix: Condition Everything on a Label 40

Chapter 16: β-VAE and Disentanglement 42
14.1 What Disentanglement Means 42
14.2 The β-VAE Fix: Weight the KL Term More Heavily 42
14.3 The Trade-off 42
Chapter 17: VQ-VAE — Discrete Latent Codes 43
15.1 Why Discrete Latents? 43
15.2 How It Works 43
15.3 Codebook Collapse 43
Chapter 18: VAEs vs. GANs vs. Diffusion Models 45
16.1 GANs (Generative Adversarial Networks) 45
16.2 Diffusion Models 45
16.3 Comparison Table 45
16.4 The Deep Connection: Latent Diffusion 46
16.5 A Note on Normalizing Flows 46
PART V — Applications and Where to Go Next 48
Chapter 19: VAEs in Robotics, RL, and Autonomous Systems 49
17.1 World Models 49
17.2 Latent Action Spaces 49
17.3 Anomaly Detection and Simulation-to-Reality Gaps 49
17.4 Where This Connects to Your Own Work 50
Chapter 20: Latent Space Interpolation and Visualization 51
Chapter 21: Where to Go From Here 52
PART VI — Advanced Topics and Evaluation 53
Chapter 22: Tighter Bounds — IWAE and Hierarchical VAEs 54
22.1 The Looseness of the Plain ELBO 54
22.2 Importance-Weighted Autoencoders (IWAE) 54
22.3 Hierarchical VAEs 54
Chapter 23: Evaluating Generative Models 55
23.1 Why Evaluation Is Hard 55
23.2 Reconstruction-Based Metrics 55
23.3 The ELBO / Log-Likelihood Estimate 55
23.4 Fréchet Inception Distance (FID) 55
23.5 Qualitative / Visual Inspection 55
23.6 Summary Checklist 56
Chapter 24: Exercises and Practice Problems 57
24.1 Conceptual Questions 57
24.2 Computation Practice 57

24.3 Design Questions 57
24.4 Solutions 58
Conceptual Questions 58
Computation Practice 58
Design Questions (Sample Reasoning) 58
Appendix A: Frequently Asked Questions 60
Is z supposed to be a single vector, or a distribution? 60
Why is the prior always N(0,1) and not something else? 60
Does a bigger latent dimension always mean a better VAE? 60
Why do VAE samples often look blurrier than GAN samples? 60
Is the ELBO ever equal to log p(x) exactly? 60
Appendix B: Notation Reference 61
Appendix C: Glossary 62
Appendix D: Further Reading 63
Appendix E: Full Formula Cheat Sheet 63
Appendix F: Index of Figures 65
Appendix G: Debugging Checklist for VAE Code 66
Symptom: Loss is NaN almost immediately 66
Symptom: Reconstructions are blurry and generic, all samples look alike 66
Symptom: Random samples from the prior look like garbage 66
Symptom: Training loss goes down but generated samples don't improve 66

Preface

This book exists to answer one question completely: **how do we go from a plain Autoencoder to a Variational Autoencoder, and why does that jump matter?** You have already worked through Transformers, so you are comfortable with neural networks, gradients, and training loops. What is new here is not neural network mechanics — it is a way of thinking about networks as *probability machines* rather than pure function approximators.

Every concept is built from scratch. We do not assume you remember graduate probability theory. Each idea is introduced with a plain-English explanation first, then a diagram, then the mathematics, and finally, where useful, a short code sketch. By the end, you will be able to explain the Evidence Lower Bound (ELBO) to someone else without notes, implement a VAE from scratch in PyTorch, and understand how the same ideas power modern systems such as latent diffusion models and world models used in robotics.

> **HOW TO READ THIS BOOK**
>
> Part I rebuilds Autoencoders from the ground up. Part II is a compact probability toolkit — only the pieces you actually need for VAEs. Part III is the core: the full derivation and implementation of the Variational Autoencoder. Part IV covers the major variants. Part V connects everything to generative modeling at large and to your own interests in robotics and autonomous systems. Part VI covers tighter bounds and evaluation, followed by worked exercises.

Mathematical Prerequisites

You do not need much beyond what a standard deep learning course (and your Transformer coursework) has already covered. This half-page is simply a checklist of the exact tools used later, so any rusty spots can be refreshed before you need them.

From Calculus

- **The chain rule** — required to follow how gradients flow backward through the encoder, the sampling step, and the decoder during backpropagation.
- **Partial derivatives** — every gradient in this book is with respect to one parameter at a time, holding others fixed.

From Linear Algebra

- **Vectors and dot products** — inputs x, latent codes z, and parameters μ, σ are all vectors.
- **Matrix-vector multiplication** — every neural network layer is fundamentally this operation plus a nonlinearity.

From Probability

- **Expectation E[·]** — a weighted average of a quantity over a distribution. Read E_{z~q}[f(z)] as "the average value of f(z), where z is drawn according to distribution q."
- **Linearity of expectation** — E[a+b] = E[a]+E[b], used repeatedly when splitting the ELBO in Chapter 9.
- **The Gaussian (Normal) distribution** — its shape, mean, and variance. Fully introduced from scratch in Chapter 5 regardless.

> **IF ANY OF THIS IS RUSTY**
>
> Do not worry. Every one of these tools is re-introduced with a plain-English explanation exactly where it is first used — this checklist is only here so you know in advance that nothing more exotic than this is required anywhere in the book.

PART I — Foundations: Rebuilding the Autoencoder

Chapter 1: What Problem Are We Actually Solving?

Before touching any equations, let's get the big picture right. Deep learning models that you have used so far — including Transformers — are mostly **discriminative**: given an input, they predict an output label, a translation, or a next token. This book is about a different family: **generative models**, whose job is to learn what the data *looks like* well enough to produce brand-new, realistic examples of it.

To generate something new, a model first needs a compact, structured way to represent what it has seen. Imagine trying to describe every photograph of a human face using only a few numbers — say, 20 numbers instead of 150,000 pixel values. If you could do that well, you could also walk through that 20-number space and generate faces that never existed. This compact numerical summary is called a **latent representation**, and the space it lives in is called **latent space**. Everything in this book is really about learning good latent spaces and learning how to sample from them intelligently.

> **KEY IDEA**
> A generative model does not memorize data. It learns the underlying *distribution* that the data was drawn from, so it can draw new samples from that same distribution.

The Autoencoder is the simplest possible way to learn a compact representation. It is not by itself a strong generative model — we will see exactly why in Chapter 4 — but it introduces every architectural piece the Variational Autoencoder (VAE) needs: an encoder, a bottleneck, and a decoder. Getting comfortable with plain Autoencoders first makes the VAE feel like a small, well-motivated upgrade rather than a wall of new notation.

1.1 Roadmap of this Book

- **Part I** — build the Autoencoder from scratch: architecture, loss, training, and its variants (denoising, sparse, contractive).
- **Part II** — a compact probability toolkit: only the ideas needed to understand VAEs (distributions, KL divergence, Bayes' rule).
- **Part III** — the core of the book: why plain Autoencoders fail as generators, and how the VAE fixes this with the ELBO, the reparameterization trick, and a full architecture + training walkthrough.
- **Part IV** — the VAE family: Conditional VAE, Beta-VAE, VQ-VAE, and how VAEs relate to GANs and Diffusion Models.

- **Part V** — applications, especially to robotics and autonomous systems (world models, latent action spaces), plus where to go next.

Chapter 2: The Plain Autoencoder

2.1 Definition

An **Autoencoder (AE)** is a neural network trained to reconstruct its own input. That's it — the target output is the input itself. This sounds trivial (why not just use an identity function?), and the trick is that we force the network through a **bottleneck**: a layer with far fewer dimensions than the input. To reconstruct the input well despite this bottleneck, the network is forced to discover and keep only the most important, compressible structure in the data.

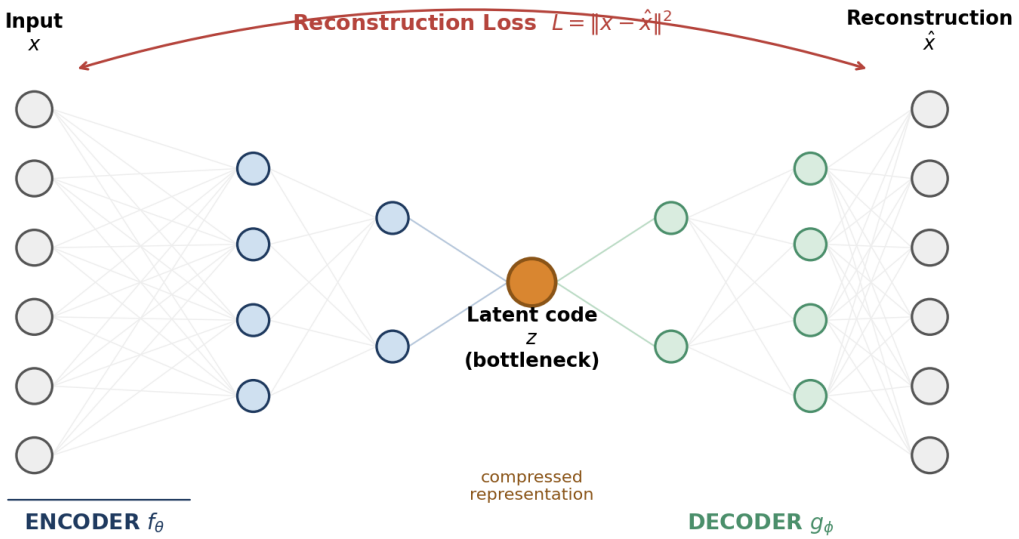


Figure 2.1 — A plain Autoencoder. The encoder compresses input x into a low-dimensional latent code z; the decoder reconstructs x̂ from z alone.

Formally, an Autoencoder has two parts:

- **Encoder** `f_θ(x) = z` — maps a high-dimensional input x (e.g. an image) to a low-dimensional code z.
- **Decoder** `g_ϕ(z) = x̂` — maps the code z back to a reconstruction x̂ that should closely match x.

Both f and g are neural networks (fully connected layers, convolutional layers, or anything else appropriate for the data), trained jointly, end to end, using ordinary backpropagation.

2.2 The Loss Function

Training simply minimizes the difference between the input and its reconstruction. For continuous data (like normalized pixel intensities), Mean Squared Error is typical:

```
L(x, x̂) = || x - x̂ ||² = Σᵢ (xᵢ - x̂ᵢ)²
```

For data that behaves like probabilities (e.g. pixels squashed into [0,1] by a sigmoid), Binary Cross-Entropy is often used instead:

$$L(x, \hat{x}) = -\Sigma_i \left[x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i) \right]$$

There is no label, no external supervision signal — the input **is** the target. This makes Autoencoders a form of **self-supervised learning**: we manufacture a supervised-looking problem (predict x from x) out of unlabeled data purely to force useful structure to emerge in the bottleneck.

2.3 A Minimal PyTorch Implementation

Here is a complete, minimal Autoencoder for 28×28 grayscale images (e.g. MNIST digits), compressed down to a 16-dimensional latent code.

```
import torch
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self, latent_dim=16):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 256), nn.ReLU(),
            nn.Linear(256, 64), nn.ReLU(),
            nn.Linear(64, latent_dim)          # bottleneck
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64), nn.ReLU(),
            nn.Linear(64, 256), nn.ReLU(),
            nn.Linear(256, 28*28), nn.Sigmoid(),
            nn.Unflatten(1, (1, 28, 28))
        )

    def forward(self, x):
        z = self.encoder(x)
        x_hat = self.decoder(z)
        return x_hat, z

model = Autoencoder(latent_dim=16)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

for x, _ in dataloader:            # labels are ignored entirely
    x_hat, z = model(x)
    loss = loss_fn(x_hat, x)
    optimizer.zero_grad()
```

```
    loss.backward()
    optimizer.step()
```

Notice two things that will matter a lot in Part III: (1) the encoder outputs a single fixed vector z — not a distribution — and (2) nothing in the loss function encourages the latent space to have any particular overall shape. Both of these will change with the VAE.

Chapter 3: Varieties of Autoencoders

The plain Autoencoder above is the baseline. In practice, several variants are used depending on the goal — mostly to prevent the network from cheating (learning a trivial identity mapping instead of useful structure) or to encourage specific properties in the latent code.

3.1 Undercomplete vs. Overcomplete Autoencoders

An Autoencoder is called **undercomplete** when the latent dimension is smaller than the input dimension — this is the standard setup, and the size gap itself is what forces compression. It is called **overcomplete** when the latent dimension is equal to or larger than the input dimension. Overcomplete Autoencoders have no natural bottleneck, so without extra constraints they can trivially learn the identity function and copy input to output without learning anything useful. To make overcomplete Autoencoders useful, we need to add explicit constraints — which is exactly what the next few variants do.

3.2 Denoising Autoencoders (DAE)

Instead of feeding the encoder a clean input, we feed it a corrupted version (e.g. pixels with random noise added, or randomly zeroed-out regions), and still ask the decoder to reconstruct the **original clean** input:

```
x̃ = corrupt(x)
z = encoder(x̃)
x̂ = decoder(z)
loss = || x - x̂ ||²      # compare to the ORIGINAL, not the corrupted input
```

This forces the network to learn robust, meaningful features rather than a superficial pixel-copy shortcut — a corrupted pixel value alone doesn't tell you what the clean value was, so the network must lean on broader context and structure to fill in gaps correctly.

3.3 Sparse Autoencoders

Here we allow a large (even overcomplete) latent layer, but add a penalty that pushes most latent units toward zero activation for any given input, so only a small subset of units are ever active at once. A common approach adds an L1 penalty on the latent activations, or a KL-divergence penalty pushing the average activation of each unit toward a small target value ρ (e.g. 0.05). This mimics how biological neurons are thought to work — sparse and specialized — and tends to produce interpretable, disentangled-ish features.

3.4 Contractive Autoencoders (CAE)

A Contractive Autoencoder adds a penalty on the sensitivity of the latent code to small changes in the input — specifically the Frobenius norm of the Jacobian of the encoder. This encourages the encoder to be locally flat: small, unimportant perturbations of the input (noise) should barely move the code, while directions that matter for reconstruction should still be preserved.

3.5 Summary Table

Variant	What changes	Why
Undercomplete AE	Latent dim < input dim	Forces compression naturally
Overcomplete AE	Latent dim ≥ input dim	Needs explicit constraints to avoid trivial copying
Denoising AE	Input corrupted before encoding	Learns robust, context-aware features
Sparse AE	Penalty on activation sparsity	Encourages specialized, interpretable units
Contractive AE	Penalty on encoder sensitivity (Jacobian)	Encourages locally stable representations

Chapter 4: Why Plain Autoencoders Cannot Generate

This chapter is the hinge of the whole book. Once you genuinely understand *why* the plain Autoencoder fails as a generator, the entire motivation for the VAE — every extra term in its loss function — becomes obvious rather than mysterious.

4.1 The Tempting (But Wrong) Idea

After training an Autoencoder, we have a decoder that can turn any latent vector z into a realistic-looking output — as long as z came from actually encoding a real input. So here's the tempting idea: what if we just pick a **random** vector z and feed it to the decoder? Wouldn't that generate a brand-new, realistic sample?

In practice this fails badly. The reconstructions look like noise, or blends of unrelated digits/objects, or simply garbage. The reason lies entirely in the shape of the latent space that a plain Autoencoder ends up learning.

4.2 The Real Problem: An Unstructured Latent Space

Nothing in the Autoencoder's loss function says anything about what the latent space as a whole should look like. The only requirement is: **each individual training point** must map to *some* z from which the decoder can reconstruct it. Between and around those encoded points, the latent space is free to do whatever minimizes the loss — which is usually a jumble of disconnected clusters, with large gaps, holes, and regions the decoder has never seen and was never trained on.

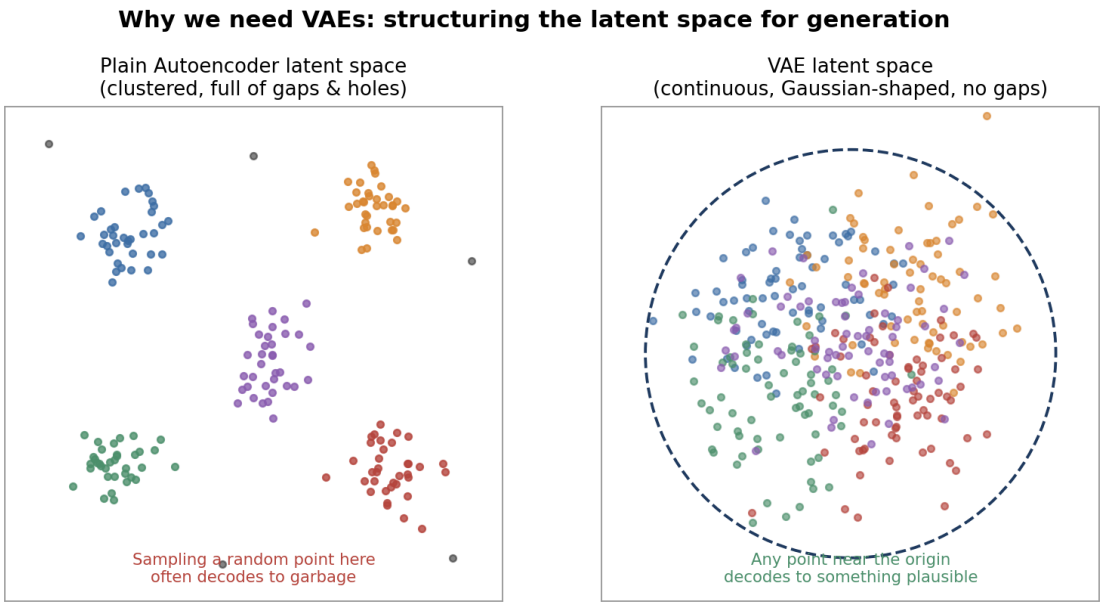


Figure 4.1 — A plain Autoencoder's latent space is scattered into unpredictable clusters with gaps (left). A well-regularized latent space (right, the VAE's) is continuous and densely filled, so any nearby point decodes to something plausible.

If you sample a random z, the odds are high you'll land in one of those untrained gaps — and the decoder, having never seen anything like it during training, produces an arbitrary, often meaningless output. This is not a bug in a particular Autoencoder implementation; it is a structural consequence of the loss function used. **We never asked the latent space to be smooth, continuous, or centered anywhere in particular — so it isn't.**

4.3 What We Actually Need

For random sampling of z to work reliably, we need the latent space to satisfy two properties:

- **Continuity** — points that are close together in latent space should decode to similar, still-plausible outputs. No sudden cliffs into garbage.
- **Completeness (no gaps)** — every point sampled from a simple, known distribution (like a standard Gaussian) should decode to something reasonable, not just the exact points seen during training.

> **THE BIG IDEA BEHIND VAES**
>
> Instead of letting the encoder map each input to a single fixed point z, force it to map each input to a small *cloud* (a probability distribution) of nearby points, and explicitly push all of these clouds toward overlapping a single, well-known reference distribution — usually a standard Gaussian N(0, 1). Overlapping clouds fill in the gaps and guarantee the space is continuous.

That single idea — encode to a distribution, not a point, and regularize that distribution toward a known shape — is the entire conceptual leap from Autoencoder to Variational Autoencoder. Everything in Part III is really just the mathematics required to make that idea precise and trainable. Before we get there, Part II builds the small amount of probability theory needed to follow the derivation comfortably.

PART II — The Probability Toolkit

You do not need a full course in probability theory to understand VAEs. You need exactly four ideas, explained here in plain language before we touch any VAE-specific notation. If you already know these cold, skim this part and jump to Part III.

Chapter 5: Random Variables and Distributions

5.1 What a Distribution Really Represents

A **probability distribution** describes how likely different outcomes are. For our purposes, think of it as a rule that assigns a "density" (a relative likelihood) to every possible value a variable could take. We will work almost exclusively with the **Gaussian (Normal) distribution**, written $N(\mu, \sigma^2)$, because it is simple, well understood, and has convenient mathematical properties that make training tractable.

A 1-D Gaussian has two parameters:

- **μ (mu)** — the mean, i.e. the center of the bell curve.
- **σ^2 (sigma squared)** — the variance, i.e. how spread out the curve is. σ itself is the standard deviation.

In a VAE, every input x will be mapped not to one point, but to its own little Gaussian bump in latent space — its own μ and σ. A **multivariate** Gaussian just repeats this independently across each of the (say, 20 or 100) latent dimensions.

5.2 Sampling

Sampling means drawing a concrete value out of a distribution. If $z \sim N(0, 1)$ (read: "z is sampled from a standard normal distribution"), most draws will land close to 0, occasionally further away, following the familiar bell-curve shape. Sampling is inherently random — running it twice gives two different values. This randomness is exactly what will cause a technical headache in Chapter 9, solved by the reparameterization trick in Chapter 10.

Chapter 6: KL Divergence

6.1 Measuring the Distance Between Two Distributions

The **Kullback-Leibler (KL) divergence** measures how different one probability distribution is from another. Given two distributions q and p over the same variable, it is defined as:

```
D_KL( q(z) || p(z) )  =  ∫ q(z) · log[ q(z) / p(z) ] dz
                      =  E_{z~q} [ log q(z) - log p(z) ]
```

In plain English: on average (weighted by how likely q thinks each value is), how much more/less likely does q consider each outcome compared to p? If q and p are identical everywhere, the KL divergence is exactly 0. The more q's shape diverges from p's, the larger the KL divergence grows — and it is always ≥ 0, never negative.

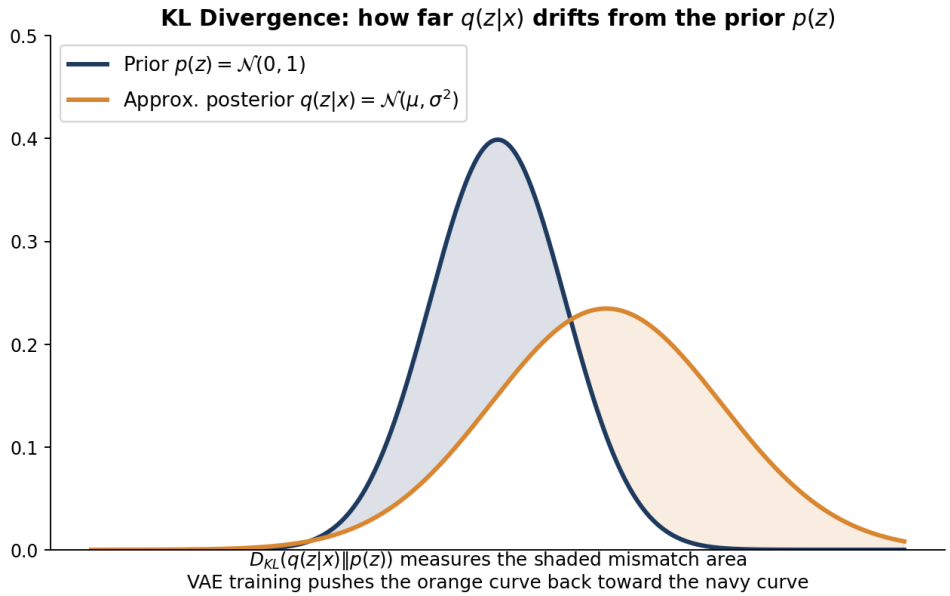


Figure 6.1 — KL divergence quantifies the mismatch area between the encoder's distribution q(z|x) and the prior p(z).

6.2 Two Important Warnings

- **It is not symmetric.** D_KL(q||p) is generally not equal to D_KL(p||q). It is a “divergence,” not a true mathematical distance.
- **It penalizes q for putting mass where p has none**, much more harshly than the reverse. This shapes how VAEs behave: the encoder's distribution is discouraged from wandering into regions the prior considers very unlikely.

6.3 The One Formula You Actually Need

When both distributions are Gaussians — specifically, when q = N(μ, σ²) and p = N(0, 1) — the KL divergence has a beautiful **closed-form solution** (no integral needs to be computed at training time):

```
D_KL( N(μ, σ²) || N(0, 1) )  =  ½ · ( μ² + σ² - log(σ²) - 1 )
```

This single line is the entire “regularization loss“ of a VAE. It is cheap to compute, easy to differentiate, and is exactly zero only when μ = 0 and σ = 1 — i.e. exactly when the encoder's distribution matches the prior perfectly. Memorize this formula; it appears directly in the training code in Chapter 11.

Chapter 7: Bayes' Rule and Latent Variable Models

7.1 Bayes' Rule Refresher

Bayes' rule relates a “forward“ probability to a “backward“ one:

```
p(z | x)  =  p(x | z) · p(z)  /  p(x)
```

- `p(z)` — the **prior**: what we believe about z before seeing any data (in our case, simply N(0,1)).
- `p(x|z)` — the **likelihood**: given a latent code z, how likely is this particular x? This is exactly what our decoder will model.
- `p(z|x)` — the **posterior**: given that we observed x, what does that tell us about z? This is what our encoder is trying to approximate.
- `p(x)` — the **evidence**: the overall probability of x, averaged over every possible z. This is the troublesome term — see below.

7.2 Latent Variable Models

A **latent variable model** assumes that every observed data point x was generated by a two-step random process: first, a latent code z is drawn from a prior p(z); then, x is generated from z via p(x|z). This is precisely the generative process we want to learn — it directly mirrors the Autoencoder's encoder/decoder split, but now every step is a probability distribution instead of a deterministic function.

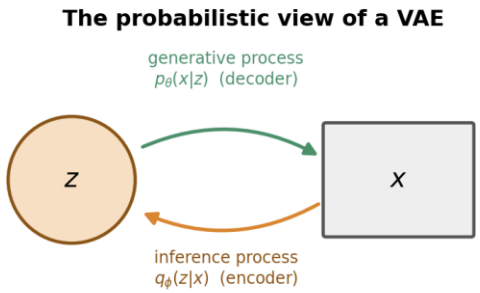


Figure 7.1 — The probabilistic view: p(x|z) is the generative (decoder) direction; q(z|x) is the inference (encoder) direction that approximates the true, intractable posterior p(z|x).

7.3 The Problem: p(x) is Intractable

To train this model properly by maximum likelihood, we would want to directly maximize p(x) for our training data. But:

```
p(x)  =  ∫ p(x | z) · p(z) dz
```

This integral sums over **every possible value of z** — an infinite, continuous space. For anything beyond toy problems, this integral has no closed-form solution and cannot be computed directly. Similarly, the true posterior p(z|x) = p(x|z)p(z)/p(x) also depends on this same impossible integral, so we cannot compute it exactly either.

THE CENTRAL OBSTACLE

We cannot compute p(x) exactly, so we cannot train the model by directly maximizing likelihood, and we cannot compute the true posterior p(z|x) either. We need a workaround — this is exactly the problem the VAE and the ELBO are built to solve, starting in the next chapter.

PART III — The Variational Autoencoder

This is the heart of the book. We now have everything we need: we know why plain Autoencoders fail as generators (Chapter 4), and we have the probability tools to fix it (Part II). This part derives the VAE objective from first principles, explains the reparameterization trick, and walks through a complete, working implementation.

Chapter 8: Setting Up the Variational Problem

8.1 Restating the Goal

We want to learn a generative model of our data: a decoder $p_\theta(x|z)$ and a simple prior $p(z) = N(0,1)$, such that sampling z from the prior and pushing it through the decoder produces realistic data. Ideally we would train this by maximizing the log-likelihood $\log p_\theta(x)$ of our training data directly. As shown in Chapter 7, this requires an intractable integral over all z.

8.2 Introducing an Approximate Posterior

Since we cannot compute the true posterior $p(z|x)$, we introduce a separate network — the encoder — to learn an approximation of it, denoted $q_\phi(z|x)$. We choose q_ϕ to be something simple and tractable: a Gaussian whose mean μ and variance σ^2 are output by a neural network that takes x as input.

```
q_ϕ(z | x)  =  N( z ;  μ_ϕ(x),  σ_ϕ(x)² )
```

This is precisely the “encode to a cloud, not a point” idea from Chapter 4, now written formally. The name Variational Autoencoder comes from this step: in statistics, approximating an intractable posterior with a simpler, tractable distribution is called Variational Inference — we are picking the best member of a simple family of distributions (Gaussians) to stand in for the true, unreachable posterior.

Chapter 9: Deriving the ELBO, Step by Step

This chapter derives the single most important equation in the book. Read it slowly — every line follows from the one before it using only algebra and the definitions from Part II.

9.1 Starting Point

We start from the log-likelihood of a single data point, and insert q_ϕ(z|x) in a way that changes nothing (multiplying and dividing by the same quantity inside an expectation is always legal):

```
log p_θ(x)
  = log p_θ(x) · ∫ q_ϕ(z|x) dz          (since ∫ q_ϕ(z|x) dz = 1)
  = ∫ q_ϕ(z|x) · log p_θ(x) dz
  = E_{z~q_ϕ(z|x)} [ log p_θ(x) ]
```

9.2 Bringing in Bayes' Rule

From Bayes' rule, p_θ(x) = p_θ(x,z) / p_θ(z|x). Substitute this in, then split the log of a fraction into a difference of logs, and again multiply/divide by q_ϕ(z|x) inside:

```
log p_θ(x)
  = E_q [ log ( p_θ(x,z) / p_θ(z|x) ) ]
  = E_q [ log p_θ(x,z) - log p_θ(z|x) ]
  = E_q [ log p_θ(x,z) - log q_ϕ(z|x) + log q_ϕ(z|x) - log p_θ(z|x) ]
  = E_q [ log p_θ(x,z) - log q_ϕ(z|x) ]  +  E_q [ log q_ϕ(z|x) - log p_θ(z|x) ]
```

Look closely at the second bracketed term. By the definition of KL divergence from Chapter 6, it is exactly D_KL(q_ϕ(z|x) || p_θ(z|x)) — the divergence between our approximate posterior and the TRUE (intractable) posterior. So we can write:

```
log p_θ(x)  =  ELBO(x)  +  D_KL( q_ϕ(z|x) || p_θ(z|x) )

where   ELBO(x)  =  E_q [ log p_θ(x,z) - log q_ϕ(z|x) ]
```

9.3 Why This is Exactly What We Need

We still cannot compute D_KL(q_ϕ(z|x) || p_θ(z|x)) directly — it involves the same intractable true posterior we were trying to avoid. **But** KL divergence is always ≥ 0 (Chapter 6). That means:

```
log p_θ(x)  ≥  ELBO(x)
```

The ELBO is a **lower bound** on the log-likelihood we actually want to maximize — hence the name Evidence Lower BOund ("evidence" is another name for p(x) in Bayesian statistics). Since the gap between log p_θ(x) and the ELBO is exactly that unreachable KL term, and KL ≥ 0 always, **maximizing the ELBO is a safe proxy**: it pushes up a guaranteed floor under the true likelihood, and as a bonus, it simultaneously drags q_ϕ(z|x) closer to the true posterior p_θ(z|x), since that gap is precisely what we are implicitly shrinking.

> **KEY IDEA**
> We cannot maximize log p(x) directly, so instead we maximize a computable lower bound on it — the ELBO. Maximizing the ELBO simultaneously improves the generative model AND makes the encoder a better approximation of the true (unreachable) posterior.

9.4 Splitting the ELBO into Two Practical Terms

The ELBO as written involves the joint p_θ(x,z) = p_θ(x|z)·p(z). Substituting this in and splitting the log again:

```
ELBO(x) = E_q [ log p_θ(x|z) + log p(z) - log q_ϕ(z|x) ]
        = E_q [ log p_θ(x|z) ]   -   E_q [ log q_ϕ(z|x) - log p(z) ]
        = E_q [ log p_θ(x|z) ]   -   D_KL( q_ϕ(z|x) || p(z) )
```

This is the form actually used in code. It has two clean, interpretable pieces:

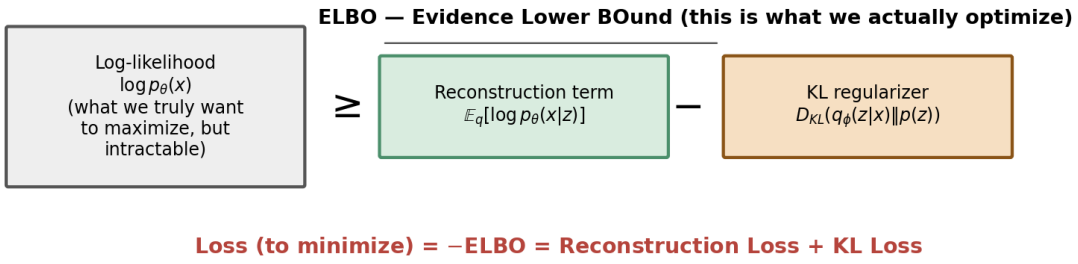


Figure 9.1 — The ELBO decomposes into a reconstruction term and a KL regularization term against the (tractable, known) prior p(z).

- **Reconstruction term** `E_q[log p_θ(x|z)]` — how well the decoder reconstructs x from a sampled z. In practice, this reduces to the same MSE or binary cross-entropy loss from Chapter 2.

- **KL term** `D_KL(q_ϕ(z|x) || p(z))` — how far the encoder's distribution strays from the prior N(0,1). This uses the exact closed-form formula from Chapter 6 — no intractable integral anywhere in sight.

Because we maximize the ELBO but frameworks like PyTorch minimize a loss, we simply flip the sign. This gives the final, famous VAE loss function:

```
L_VAE(x) = Reconstruction_Loss(x, x̂) + D_KL( N(μ,σ²) || N(0,1) )
         = || x - x̂ ||²  +  ½ · Σ ( μ² + σ² - log(σ²) - 1 )
```

Every symbol in this line has now been individually derived and explained. This is the full VAE loss, with nothing hidden.

Chapter 10: The Reparameterization Trick

10.1 The Problem: Sampling is Not Differentiable

Training requires backpropagating gradients from the loss, through the sampled z, back into the encoder's parameters (μ and σ, which depend on θ/ϕ). But z is produced by a **random sampling** operation, $z \sim N(\mu, \sigma^2)$. Sampling is not a differentiable function of μ and σ in the ordinary sense — you cannot ask "how does this random draw change if I nudge μ slightly?" using standard calculus, because randomness has no derivative.

10.2 The Fix: Move the Randomness Outside

The trick is a simple algebraic rewrite. Instead of sampling z directly from $N(\mu,\sigma^2)$, we sample a fixed-distribution noise variable $\varepsilon \sim N(0,1)$ (which does not depend on any learnable parameters), and compute z deterministically from it:

```
ε ~ N(0, 1)                (sample this — no gradient needed through this step)
z = μ + σ · ε              (deterministic function of μ, σ, and ε)
```

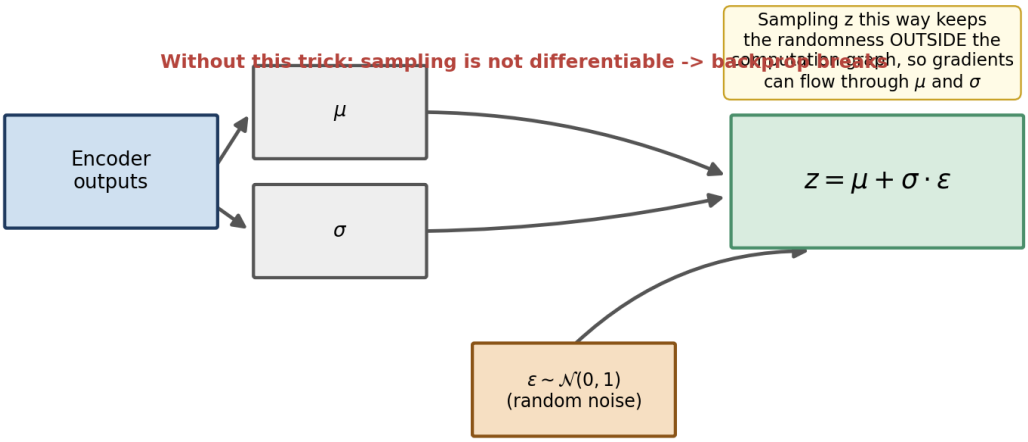


Figure 10.1 — The reparameterization trick moves the random sampling step outside the computation graph, so gradients can flow cleanly through μ and σ.

Now z is an ordinary differentiable function of μ and σ (multiplication and addition), with all the randomness isolated inside ε, which we never need to differentiate through. Gradients flow: loss → z → μ, σ → encoder weights, exactly as in a normal feed-forward network. This one-line trick is what makes end-to-end training of VAEs by ordinary backpropagation possible at all.

WHY THIS MATTERS

Without the reparameterization trick, we would need much higher-variance gradient estimators (like REINFORCE / score-function estimators) that are slow and unstable to train with. The reparameterization trick reduces VAE training to something as simple as training any other feed-forward network.

Chapter 11: Full VAE Architecture and Implementation

11.1 The Complete Picture

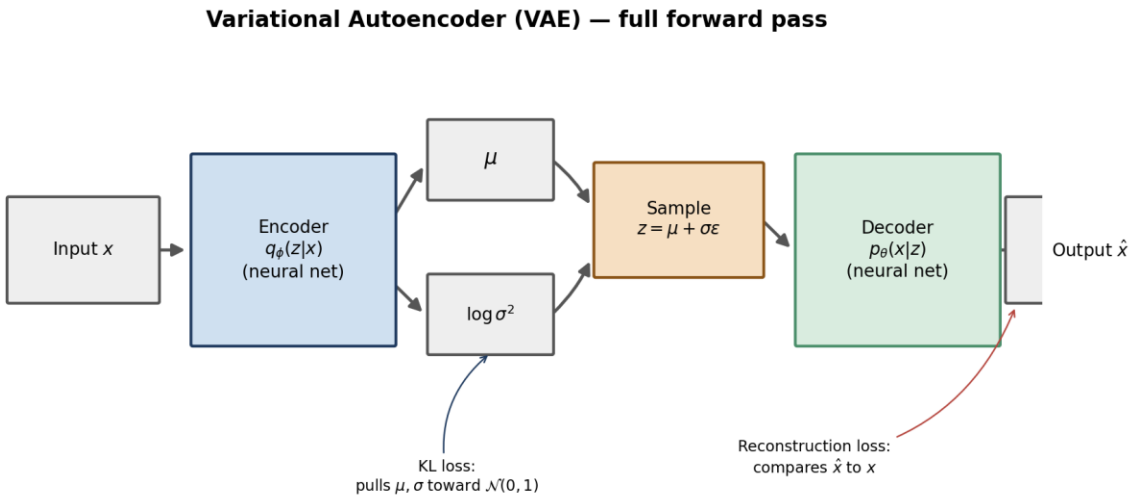


Figure 11.1 — The complete VAE forward pass: encoder produces μ and logσ², we sample z via reparameterization, and the decoder reconstructs x̂.

Compare this to the plain Autoencoder diagram in Chapter 2 (Figure 2.1). The architecture is almost identical — same encoder shape, same decoder shape — with exactly two differences: (1) the encoder now outputs two vectors, μ and logσ², instead of one; and (2) there is a sampling step (via the reparameterization trick) between encoder and decoder. Everything else you already know about training neural networks still applies.

11.2 A Note on logσ²

In code, the encoder typically outputs logσ² rather than σ or σ² directly. This is a numerical-stability trick: σ² must always be positive, but a raw neural network output can be any real number. Working in log-space removes the positivity constraint entirely (any real number is a valid logσ²), and we simply exponentiate when we need the actual standard deviation: σ = exp(0.5 · logσ²).

11.3 Full PyTorch Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class VAE(nn.Module):
    def __init__(self, latent_dim=16):
        super().__init__()
        # Shared encoder trunk
        self.enc_trunk = nn.Sequential(
```

```
            nn.Flatten(),
            nn.Linear(28*28, 256), nn.ReLU(),
            nn.Linear(256, 64), nn.ReLU(),
        )
        self.fc_mu     = nn.Linear(64, latent_dim)
        self.fc_logvar = nn.Linear(64, latent_dim)

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64), nn.ReLU(),
            nn.Linear(64, 256), nn.ReLU(),
            nn.Linear(256, 28*28), nn.Sigmoid(),
            nn.Unflatten(1, (1, 28, 28))
        )

    def encode(self, x):
        h = self.enc_trunk(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)          # ε ~ N(0,1), same shape as std
        return mu + std * eps                 # z = μ + σ·ε

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        x_hat = self.decoder(z)
        return x_hat, mu, logvar

def vae_loss(x, x_hat, mu, logvar):
    recon = F.binary_cross_entropy(x_hat, x, reduction="sum")
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon + kl, recon, kl

model = VAE(latent_dim=16)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for x, _ in dataloader:
    x_hat, mu, logvar = model(x)
    loss, recon, kl = vae_loss(x, x_hat, mu, logvar)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Notice the KL term in code: `-0.5 * sum(1 + logvar - mu^2 - exp(logvar))`. This is exactly the closed-form formula from Chapter 6, rewritten in terms of $\log\sigma^2$ instead of σ^2 (since $\exp(\text{logvar}) = \sigma^2$). Every line of this training loop maps directly back to a derivation step from Chapter 9.

11.4 Generating New Samples After Training

Once trained, generation is trivial — exactly the naive idea from Chapter 4, except now it actually works because the latent space is continuous and centered at the origin:

```
model.eval()
with torch.no_grad():
    z = torch.randn(16, latent_dim)   # sample directly from the prior N(0,1)
    samples = model.decoder(z)        # decode straight to new, original images
```

Chapter 12: Training Dynamics and Common Failure Modes

12.1 What Healthy Training Looks Like

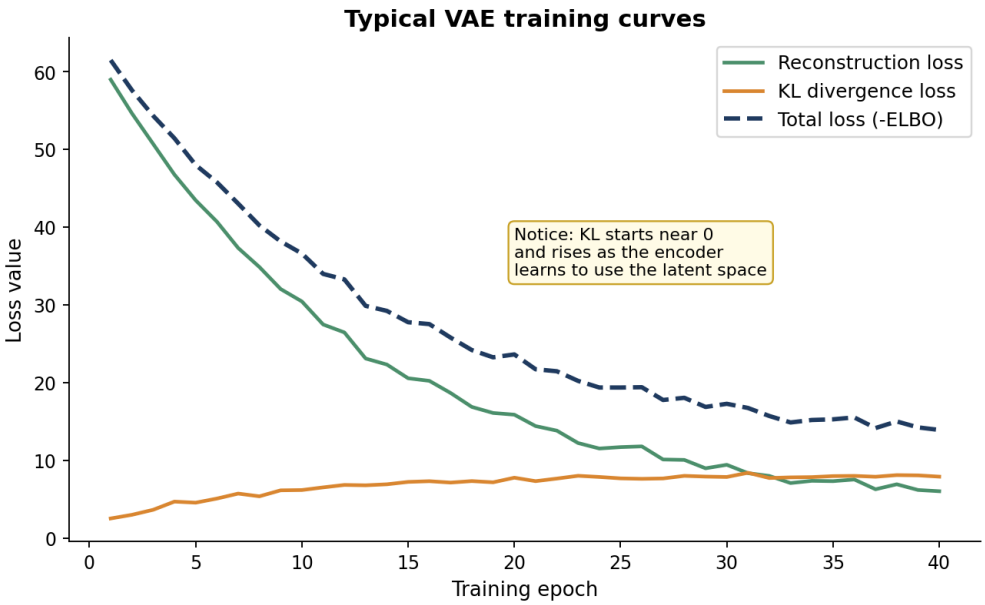


Figure 12.1 — Typical VAE training curves: reconstruction loss falls steadily, while the KL term rises from near-zero as the encoder learns to actually use the latent space.

Early in training, the encoder has not yet learned anything useful, so it tends to output $\mu \approx 0$ and $\sigma \approx 1$ for every input — exactly matching the prior, which trivially minimizes the KL term but produces poor (blurry, generic) reconstructions. As training proceeds, the encoder starts spreading different inputs to different regions of latent space to improve reconstruction, and the KL term rises accordingly. A healthy run settles into a balance where both terms are meaningfully non-zero.

12.2 The Reconstruction vs. Regularization Trade-off

The two loss terms actively pull in opposite directions. The reconstruction term wants the encoder to spread inputs far apart in latent space (easier to tell them apart, easier to reconstruct precisely). The KL term wants every input squeezed toward the same N(0,1) blob. The final latent space is a compromise between these two pressures — which is exactly the source of the smooth, overlapping structure that makes generation possible (Figure 4.1, right panel).

12.3 Posterior Collapse

A common failure mode, especially with a powerful decoder (e.g. a large autoregressive one) or a KL term weighted too strongly: the decoder learns to ignore z almost entirely and reconstructs "average-looking" outputs from a nearly-fixed default it can produce on its own. The encoder responds by collapsing $\sigma \rightarrow 1$ and

μ → 0 for every input, since the KL loss then costs essentially nothing. Reconstructions become blurry and generic, and the latent code stops encoding meaningful information at all.

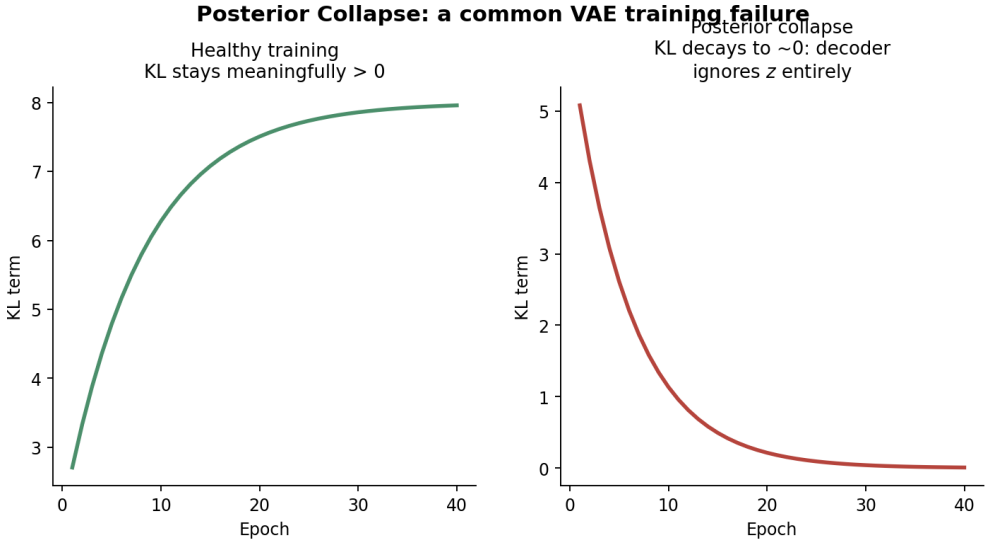


Figure 12.2 — In posterior collapse (right), the KL term decays toward zero because the encoder gives up trying to encode useful information.

12.4 Common Fixes

- **KL annealing (warm-up)** — start training with the KL term's weight at 0 and gradually increase it to 1 over the first several epochs, so the model first learns to reconstruct well before being forced to regularize.
- **β-VAE weighting** — introduce a weight β on the KL term (covered fully in Chapter 14) and tune it below 1 to reduce collapse risk, at some cost to latent-space regularity.
- **Free bits** — only penalize KL divergence beyond a small threshold per latent dimension, guaranteeing each dimension is allowed to carry at least a minimum amount of information before being penalized.
- **Weaker decoders / limiting decoder context** — preventing the decoder from being powerful enough to ignore z in the first place.

Chapter 13: A Complete Worked Numerical Example

Equations are easiest to trust once you have pushed real numbers through them. This chapter takes one tiny, artificial example — a 3-dimensional input compressed to a 2-dimensional latent code — and computes every quantity from Chapters 9–11 by hand.

13.1 Setup

Suppose our input is x = [0.9, 0.1, 0.8] (three numbers, already scaled to [0,1] as if they were three pixel intensities). Suppose that, after passing x through the encoder trunk, the two output heads produce:

```
μ       = [ 0.40, -0.30 ]
logσ²   = [ -0.20,  0.10 ]
```

First, recover σ from logσ² using σ = exp(0.5 · logσ²):

```
σ₁ = exp(0.5 × -0.20) = exp(-0.10) ≈ 0.9048
σ₂ = exp(0.5 ×  0.10) = exp( 0.05) ≈ 1.0513
```

13.2 Sampling z via the Reparameterization Trick

Draw noise ε from N(0,1) — say (for this worked example) we happen to draw ε = [0.50, -1.20]. Then:

```
z₁ = μ₁ + σ₁·ε₁ = 0.40 + (0.9048 ×  0.50) = 0.40 + 0.4524 = 0.8524
z₂ = μ₂ + σ₂·ε₂ = -0.30 + (1.0513 × -1.20) = -0.30 - 1.2616 = -1.5616

z = [ 0.8524, -1.5616 ]
```

This z is now fed into the decoder. Suppose the decoder (whatever its internal layers) produces a reconstruction:

```
x̂ = [ 0.85, 0.15, 0.78 ]
```

13.3 Computing the Reconstruction Loss

Using Mean Squared Error (sum form, as in Chapter 2):

```
Recon = (0.90-0.85)² + (0.10-0.15)² + (0.80-0.78)²
      = (0.05)² + (-0.05)² + (0.02)²
      = 0.0025 + 0.0025 + 0.0004
      = 0.0054
```

13.4 Computing the KL Term

Using the closed-form Gaussian KL formula from Chapter 6, applied per-dimension and summed:

```
D_KL = ½ · Σᵢ ( μᵢ² + σᵢ² - logσᵢ² - 1 )

Dimension 1: ½ × ( 0.40² + 0.9048² - (-0.20) - 1 )
           = ½ × ( 0.16 + 0.8187 + 0.20 - 1 )
           = ½ × ( 0.1787 )  =  0.0894

Dimension 2: ½ × ( (-0.30)² + 1.0513² - 0.10 - 1 )
           = ½ × ( 0.09 + 1.1052 - 0.10 - 1 )
           = ½ × ( 0.0952 )  =  0.0476

D_KL(total) = 0.0894 + 0.0476 = 0.1370
```

13.5 The Final Loss

```
L_VAE = Recon + D_KL = 0.0054 + 0.1370 = 0.1424
```

This single number, 0.1424, is what gets backpropagated: through the decoder, through the $z = \mu + \sigma \cdot \epsilon$ sampling step, and into μ and $\log\sigma^j$ in the encoder — exactly the path described in Chapter 10. Notice that the KL term (0.137) currently dominates the reconstruction term (0.0054) in this toy example; in early training, this imbalance is completely normal and is exactly the pattern shown in Figure 12.1.

> **TRY IT YOURSELF**
>
> Recompute this example with $\mu = [0,0]$ and $\log\sigma^2 = [0,0]$ (i.e. matching the prior exactly) and confirm the KL term comes out to precisely 0. This is a good sanity check to run on your own code as well.

Chapter 14: Convolutional VAEs for Real Image Data

14.1 Why Flatten-and-Linear Doesn't Scale

The MLP-based VAE in Chapter 11 works fine for small 28×28 grayscale digits, but flattening a realistic RGB image (e.g. 256×256×3) into a single vector throws away all spatial structure and requires enormous fully-connected layers. In practice, real image VAEs use convolutional layers in the encoder and transposed convolutional (or upsampling) layers in the decoder, exactly as in standard CNN architectures you already know from computer vision work.

14.2 A Convolutional Encoder/Decoder Sketch

```
class ConvVAE(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(3, 32, 4, stride=2, padding=1), nn.ReLU(),   # 128->64
            nn.Conv2d(32, 64, 4, stride=2, padding=1), nn.ReLU(),  # 64->32
            nn.Conv2d(64, 128, 4, stride=2, padding=1), nn.ReLU(), # 32->16
            nn.Flatten(),
        )
        self.fc_mu     = nn.Linear(128*16*16, latent_dim)
        self.fc_logvar = nn.Linear(128*16*16, latent_dim)
        self.fc_dec    = nn.Linear(latent_dim, 128*16*16)

        self.dec = nn.Sequential(
            nn.Unflatten(1, (128, 16, 16)),
            nn.ConvTranspose2d(128, 64, 4, stride=2, padding=1), nn.ReLU(),  # 16->32
            nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1), nn.ReLU(),   # 32->64
            nn.ConvTranspose2d(32, 3, 4, stride=2, padding=1), nn.Sigmoid(), # 64->128
        )

    def encode(self, x):
        h = self.enc(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def decode(self, z):
        return self.dec(self.fc_dec(z))

    def forward(self, x):
        mu, logvar = self.encode(x)
        std = torch.exp(0.5*logvar)
        z = mu + std * torch.randn_like(std)
        return self.decode(z), mu, logvar
```

The loss function, reparameterization trick, and training loop are completely unchanged from Chapter 11 — only the encoder and decoder internals differ. This is a general pattern worth internalizing: the VAE *framework*

(encode to a distribution, sample, regularize with KL, reconstruct) is architecture-agnostic. You can swap in convolutional layers, Transformer blocks, or graph neural networks depending on the structure of your data, and everything from Chapters 9 and 10 still applies unchanged.

14.3 Practical Tips for Image VAEs

- Use **strided convolutions** for downsampling in the encoder rather than pooling layers, so spatial information reduction is learned rather than fixed.
- Match encoder and decoder depths symmetrically so spatial dimensions line up cleanly on the way back up.
- Batch Normalization or Group Normalization between conv layers generally stabilizes training significantly.
- For higher resolution images, consider a VAE that compresses to a *spatial* latent grid (e.g. 16×16×4) rather than a single flat vector — this is exactly the approach used in Latent Diffusion / Stable Diffusion, mentioned in Chapter 16.

PART IV — The VAE Family and Its Neighbors

With the core VAE fully derived, this part covers the most important variants, each fixing a specific limitation of the vanilla model, plus a comparison against the other major generative modeling families.

Chapter 15: Conditional VAE (CVAE)

13.1 The Limitation of a Plain VAE

A vanilla VAE lets you generate a random, plausible sample — but you cannot control what you get. If trained on handwritten digits 0–9, sampling z ~N(0,1) might give you a 7 or a 3, with no way to request a specific digit.

13.2 The Fix: Condition Everything on a Label

The Conditional VAE simply feeds an extra piece of information y (a class label, an attribute, a caption embedding — anything) into **both** the encoder and the decoder alongside x and z respectively:

```
Encoder:  q_ϕ(z | x, y)   – concatenate y onto x before/inside the encoder
Decoder:  p_θ(x | z, y)   – concatenate y onto z before/inside the decoder
```

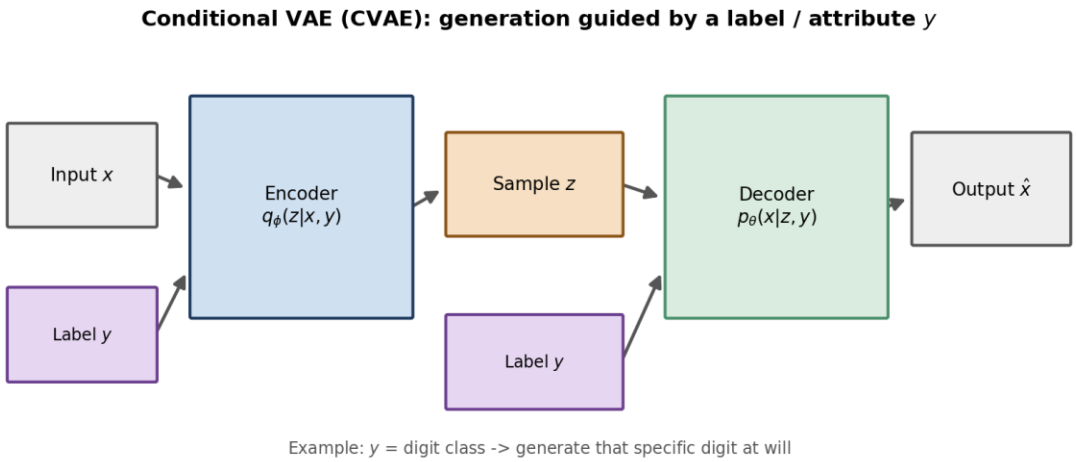


Figure 13.1 — A Conditional VAE threads the label y through both encoder and decoder, so generation can be steered.

The loss function is unchanged in form — reconstruction + KL — simply with everything conditioned on y as well. At generation time, you now pick both a random z **and** the label y you want, and the decoder produces a sample of that specific class:

```
z = torch.randn(1, latent_dim)
y = one_hot(digit=7)              # explicitly request a 7
sample = model.decoder(torch.cat([z, y], dim=1))
```

This pattern — conditioning a generative model on extra side information — reappears constantly in modern systems, including text-to-image diffusion models, where the “label” is a text embedding instead of a one-hot class vector.

Chapter 16: β-VAE and Disentanglement

14.1 What Disentanglement Means

In an ideal latent space, each individual latent dimension would correspond to one interpretable, independent factor of variation in the data — e.g. one dimension controls digit rotation, another controls stroke thickness, a third controls slant — with no dimension tangled up with multiple factors at once. A vanilla VAE has no explicit pressure toward this; dimensions often end up entangled, each affecting several visual attributes simultaneously.

14.2 The β-VAE Fix: Weight the KL Term More Heavily

β-VAE introduces a single scalar hyperparameter β that multiplies the KL term in the loss:

```
L_β-VAE(x) = Reconstruction_Loss(x, x̂)  +  β · D_KL( q_ϕ(z|x) || p(z) )          (β > 1)
```

Setting β > 1 pushes the encoder's distributions even more strongly toward matching the simple, factorized (independent-per-dimension) prior N(0,1). Because the prior treats every latent dimension as statistically independent, forcing q closer to it discourages the encoder from spreading a single real-world factor of variation across multiple correlated dimensions — nudging the model toward one-factor-per-dimension representations.

14.3 The Trade-off

Increasing β improves disentanglement but comes at a direct cost to reconstruction quality — the stronger regularization pressure leaves less capacity for the encoder to preserve fine detail. Choosing β is therefore a deliberate trade-off between interpretability of the latent space and raw output fidelity, tuned per application.

Chapter 17: VQ-VAE — Discrete Latent Codes

15.1 Why Discrete Latents?

Some kinds of data are naturally more discrete than continuous — language is made of discrete tokens, and even images often have a natural "vocabulary" of recurring textures and parts. VQ-VAE (Vector-Quantized VAE) replaces the continuous Gaussian latent space with a **discrete codebook** of learned vectors.

15.2 How It Works

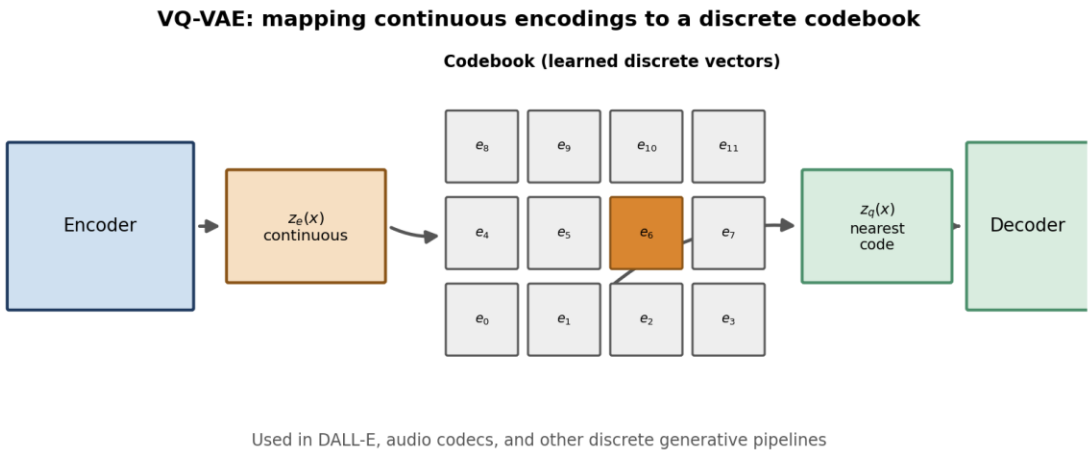


Figure 15.1 — VQ-VAE snaps each continuous encoder output to its nearest vector in a learned, finite codebook.

- The encoder produces a continuous vector z_e(x), exactly like a normal Autoencoder.
- A **codebook** of K learned embedding vectors {e_1, e_2, ... , e_K} is maintained.
- z_e(x) is replaced by its **nearest neighbor** in the codebook, z_q(x) = e_k where k = argmin_i ‖z_e(x) - e_i‖.
- The decoder reconstructs x from this quantized z_q(x), never seeing the original continuous vector.

Because the argmin/nearest-neighbor step is not differentiable, VQ-VAE uses a straight-through gradient estimator: on the backward pass, gradients from the decoder are copied straight back to z_e(x) as if quantization had not happened. There is no KL term at all in the standard VQ-VAE loss — instead, extra terms pull the codebook vectors toward the encoder outputs, and vice versa.

VQ-VAE's discrete codes are exactly what made systems like DALL-E (original version) and many neural audio codecs practical: once you have a small, finite vocabulary of latent codes, you can train a separate, powerful sequence model (like a Transformer!) directly on sequences of those discrete codes — tying this whole book back to where your journey started.

15.3 Codebook Collapse

VQ-VAE has its own characteristic failure mode, analogous to posterior collapse in a regular VAE: **codebook collapse**, where only a small handful of the K codebook vectors ever get used, and the rest sit permanently unused ("dead") because nothing in the input distribution ever routes to them. This wastes model capacity and can visibly reduce output diversity. Common fixes include periodically re-initializing dead codebook vectors to the encoder outputs of randomly chosen training examples, and using an exponential-moving-average update rule for the codebook (as in the original VQ-VAE paper) rather than plain gradient descent, which tends to keep code usage more balanced.

Chapter 18: VAEs vs. GANs vs. Diffusion Models

VAEs are one member of a broader family of generative models. Understanding how they compare to their two biggest rivals — GANs and Diffusion Models — will help you choose the right tool and understand the current state of the field.

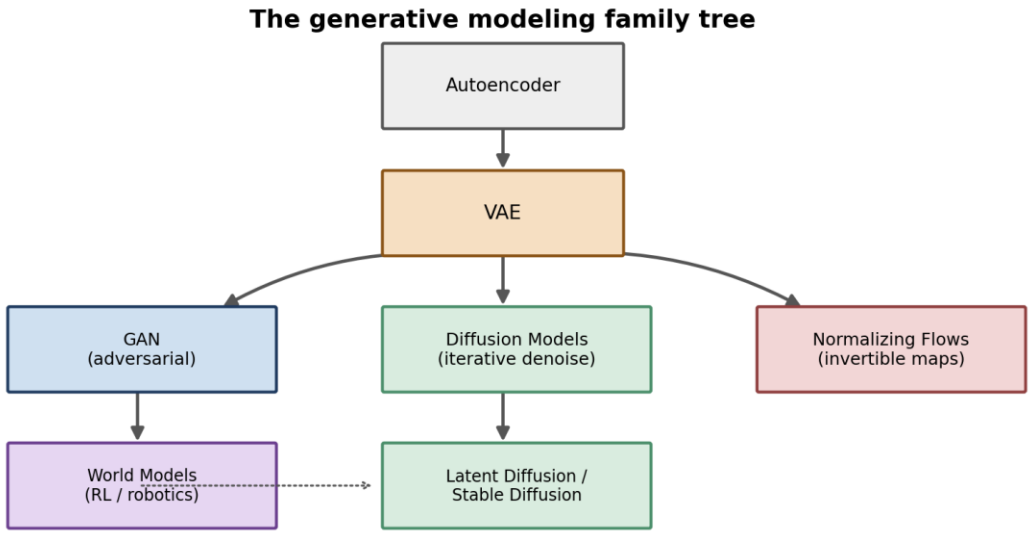


Figure 16.1 — The generative modeling family tree: VAEs, GANs, Diffusion Models, and Normalizing Flows all solve the same underlying problem with different mechanisms.

16.1 GANs (Generative Adversarial Networks)

A GAN trains two networks against each other: a **generator** that tries to produce realistic fake samples from random noise, and a **discriminator** that tries to tell real samples from fake ones. Both improve through this adversarial competition. GANs tend to produce sharper, more photorealistic samples than vanilla VAEs, but are notoriously unstable to train (mode collapse, vanishing gradients) and provide no explicit, interpretable latent-space encoder by default.

16.2 Diffusion Models

A Diffusion Model learns to reverse a gradual noising process: starting from pure noise, it iteratively denoises, step by step, until a clean sample emerges. This iterative refinement (often 20–1000 steps) tends to produce very high-quality, diverse samples, and training is comparatively stable — which is why diffusion models power most state-of-the-art image and video generators today. The trade-off is slower generation (many sequential steps) compared to a VAE's single decoder pass.

16.3 Comparison Table

Property	VAE	GAN	Diffusion
Training stability	High	Low – Medium	High
Sample sharpness	Medium (can be blurry)	High	Very High
Sample diversity	High	Medium (mode collapse risk)	High
Explicit latent encoder	Yes, built-in	No, not by default	Approximate, via inversion
Generation speed	Fast (one pass)	Fast (one pass)	Slow (many steps)
Likelihood estimate	Lower-bound (ELBO)	None	Lower-bound (ELBO-like)

16.4 The Deep Connection: Latent Diffusion

Modern systems like Stable Diffusion actually **combine** VAEs and diffusion models. An Autoencoder (specifically, a VAE-style encoder/decoder) first compresses images into a smaller latent space; the slow, expensive diffusion process then runs entirely inside that compact latent space instead of on raw pixels, making generation dramatically faster; finally the VAE's decoder expands the result back to a full-resolution image. Every concept from Part III of this book — encoder, decoder, latent space, reconstruction loss — is directly reused inside these state-of-the-art systems.

16.5 A Note on Normalizing Flows

A fourth family, briefly mentioned in Figure 16.1, is worth a short explanation here since it completes the picture. A **Normalizing Flow** builds a generative model out of a chain of invertible, differentiable transformations applied to a simple base distribution (e.g. N(0,1)). Because every transformation is exactly invertible, the exact log-likelihood log p(x) can be computed directly, with no lower bound or approximation needed — unlike the VAE's ELBO. The trade-off is architectural: every layer must be restricted to an invertible form, which limits design flexibility compared to the free-form encoder/decoder networks used in a VAE.

Model family	Likelihood	Key mechanism
VAE	Lower bound (ELBO)	Encoder/decoder + KL regularization
GAN	Not estimated	Adversarial generator/discriminator game
Diffusion	Lower bound (ELBO-like)	Iterative learned denoising
Normalizing Flow	Exact	Chain of invertible transformations

PART V — Applications and Where to Go Next

Chapter 19: VAEs in Robotics, RL, and Autonomous Systems

Given your background in robotics simulation, computer vision, and autonomous systems, this chapter connects everything above directly to work you are likely to touch soon.

17.1 World Models

A core challenge in reinforcement learning for robotics is that raw sensor input (camera images, LiDAR point clouds) is extremely high-dimensional, making it expensive to plan or learn policies directly on it. **World Models** address this by using a VAE (or a similar encoder) to compress each observation into a small latent vector z, and then learning the environment's dynamics — how z evolves over time given an action — entirely inside this compact latent space, rather than in raw pixel space.

- A **vision model** (VAE encoder) compresses each camera frame into a latent vector z.
- A **memory / dynamics model** (often a recurrent network) predicts how z evolves given the robot's action.
- A **controller / policy** operates entirely on this small latent representation to choose actions, which is far cheaper than operating on raw images.

This is the same encoder-decoder structure from Part III, repurposed: instead of just reconstructing images, the latent space now also has to be predictable and useful for control, which is exactly the kind of continuous, structured space a VAE (rather than a plain Autoencoder) is built to produce.

17.2 Latent Action Spaces

For robots with complex, high-dimensional action spaces (e.g. a humanoid with dozens of joints), directly learning a control policy over the raw action space can be difficult. A VAE-style encoder can instead be trained over sequences of actions or full trajectories, learning a smaller, smoother **latent action space**. A policy can then output a single low-dimensional latent action, which a decoder expands into a full, coordinated joint-space command — this tends to produce smoother, more physically plausible motion than optimizing the raw action space directly.

17.3 Anomaly Detection and Simulation-to-Reality Gaps

Because a trained VAE reconstructs in-distribution data well but reconstructs unfamiliar / out-of-distribution data poorly, the reconstruction error itself becomes a useful anomaly signal — useful for detecting sensor faults, unexpected obstacles, or scenarios that differ meaningfully from what a robot was trained on in simulation. This connects directly to sim-to-real transfer: a VAE trained on simulated sensor data can help flag when real-world inputs look meaningfully different from anything seen in simulation.

17.4 Where This Connects to Your Own Work

Your VLA (Vision-Language-Action) architecture explorations, your ROS 2 / Gazebo simulation projects, and your interest in computer vision all intersect directly with these ideas: a VAE-style latent representation is frequently the bridge between raw perception (cameras, point clouds) and the planning / control stack, exactly the kind of pipeline used in modern drone and humanoid robotics research.

Chapter 20: Latent Space Interpolation and Visualization

One of the most convincing demonstrations that a VAE has learned a genuinely continuous, structured latent space is **interpolation**: pick two latent codes z_A and z_B (for example, by encoding two real images), and walk in a straight line between them.

```
z_A = model.encode(image_A)[0]      # take the mean μ as a point estimate
z_B = model.encode(image_B)[0]

for alpha in torch.linspace(0, 1, steps=8):
    z_interp = (1 - alpha) * z_A + alpha * z_B
    frame = model.decoder(z_interp)     # smooth, meaningful morph
```

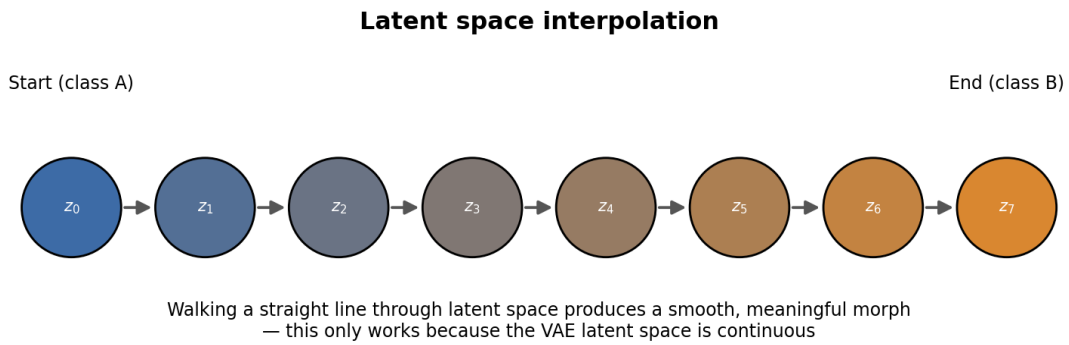


Figure 18.1 — Walking a straight line through latent space, from one encoded example to another, produces a smooth and plausible morph — only possible because the space is continuous.

If you tried this same experiment with a plain Autoencoder's latent space (Chapter 4), the walk would very likely pass through untrained gaps and produce garbage frames partway through. The fact that the VAE walk stays plausible the whole way is direct, visual proof of everything derived mathematically in Chapter 9.

Chapter 21: Where to Go From Here

You now have the complete conceptual and mathematical foundation of latent-variable generative modeling. Here is a suggested order for continuing:

1. **Implement a VAE from scratch** on MNIST or Fashion-MNIST using the code in Chapter 11. Actually training one and inspecting the latent space yourself will cement every idea in Part III far better than reading alone.
2. **Implement β-VAE** on the same dataset and visually compare disentanglement at $\beta = 1$, 4, and 10.
3. **Study GANs** (Chapter 16) — implement a simple DCGAN and compare its training stability and sample sharpness to your VAE.
4. **Study Diffusion Models** — the DDPM (Denoising Diffusion Probabilistic Models) paper is the natural next read; its loss function is derived using the exact same ELBO machinery as Chapter 9, generalized to many noising steps instead of one.
5. **Read about World Models** (Ha & Schmidhuber, 2018) for the direct robotics/RL connection covered in Chapter 19.
6. **Explore Normalizing Flows** as a third alternative: instead of a lower bound (ELBO) or an adversarial game, Normalizing Flows use invertible neural networks to compute exact likelihoods.

> **CLOSING THOUGHT**
>
> Every generative model you will encounter from here — GANs, Diffusion, Flow-based models, and their combinations — is answering the same fundamental question this book started with: how do we represent data compactly enough, and smoothly enough, to generate brand-new and realistic examples of it? You now understand one complete, rigorous answer to that question from the ground up.

PART VI — Advanced Topics and Evaluation

This final technical part covers two things every practitioner eventually runs into: tighter variational bounds than the plain ELBO, and how to actually measure whether a generative model is good.

Chapter 22: Tighter Bounds — IWAE and Hierarchical VAEs

22.1 The Looseness of the Plain ELBO

Recall from Chapter 9 that the ELBO is a lower bound on log p(x), and the gap between them is D_KL(q_ϕ(z|x) || p_θ(z|x)) — a gap that shrinks as the encoder's approximate posterior gets closer to the true posterior. If q is a simple diagonal Gaussian but the true posterior is more complex (e.g. multi-modal), that gap can remain stubbornly large no matter how well we optimize. Two major extensions address this.

22.2 Importance-Weighted Autoencoders (IWAE)

Instead of drawing a single sample z from q_ϕ(z|x), IWAE draws K independent samples z_1, ..., z_K and forms a tighter bound using a weighted average:

```
L_IWAE,K(x) = E [ log ( (1/K) × Σ□ p_θ(x,z□) / q_ϕ(z□|x) ) ]
```

As K increases, this bound provably gets tighter and tighter, approaching log p(x) itself in the limit of infinite samples. With K=1, this reduces exactly to the standard ELBO from Chapter 9 — so IWAE is a strict generalization, not a different idea. The cost is K times more compute per training step, since the encoder and decoder must process K samples instead of one.

22.3 Hierarchical VAEs

A hierarchical (or “ladder”) VAE uses multiple stacked layers of latent variables, z_1, z_2, ..., z_L, instead of one, each conditioned on the layer above it:

```
p_θ(x, z₁, ..., z_L) = p(z_L) · p_θ(z_{L-1}|z_L) · ... · p_θ(z₁|z₂) · p_θ(x|z₁)
```

Higher layers (z_L) tend to capture coarse, global structure (e.g. overall object shape), while lower layers (z_1) capture fine detail. Modern high-performing VAE variants — such as NVAE and Very Deep VAEs (VDVAE) — use dozens of these stacked latent layers and produce dramatically sharper samples than a single-layer VAE, while keeping the same core ELBO-based training machinery from Chapter 9, simply applied recursively layer by layer.

Chapter 23: Evaluating Generative Models

23.1 Why Evaluation Is Hard

Unlike a classifier, where accuracy is unambiguous, judging a generative model means judging the entire *distribution* it has learned — an inherently fuzzier target. A model that memorizes and replays training examples exactly would get a perfect reconstruction loss while being completely useless as a generator, so no single number tells the whole story.

23.2 Reconstruction-Based Metrics

The simplest check: measure the reconstruction loss (Chapter 2/9) on a held-out validation set the model has never seen. This tells you how well the model compresses and decompresses real data, but says nothing about the quality of *newly generated* (not reconstructed) samples.

23.3 The ELBO / Log-Likelihood Estimate

The ELBO itself (or a tighter IWAE bound from Chapter 22) on held-out data gives an estimate of how well the model explains unseen data, expressed in the same units as the training objective. Lower loss (higher ELBO) on validation data generally indicates a better generative model, though it does not always correlate perfectly with perceptual sample quality.

23.4 Fréchet Inception Distance (FID)

The most widely used metric for image generators. FID passes both real images and generated images through a pretrained image classifier (Inception network), extracts feature vectors from an intermediate layer, and measures the distance between the two resulting feature distributions (modeled as Gaussians) using a formula related to KL divergence in spirit:

```
FID = || μ_real - μ_fake ||² + Tr( Σ_real + Σ_fake - 2·(Σ_real · Σ_fake)^(1/2) )
```

Lower FID means the generated images' feature statistics more closely match real images' feature statistics. FID is widely used exactly because it captures both quality (realism) and diversity (mode coverage) in a single number, unlike raw pixel-wise reconstruction loss.

23.5 Qualitative / Visual Inspection

Never skip this step. Always look directly at: (1) reconstructions of held-out data, (2) fresh random samples from the prior, and (3) a latent-space interpolation grid (Chapter 20). Blurriness, mode collapse (all samples

looking alike), or implausible artifacts are usually obvious to the eye well before any single metric clearly flags them.

23.6 Summary Checklist

Check	What it tells you
Held-out reconstruction loss	How well the model compresses/decompresses real data
Held-out ELBO / IWAE bound	Overall estimated data likelihood under the model
FID (images)	Realism and diversity of freshly generated samples
Latent interpolation (visual)	Whether the latent space is smooth and continuous
Random prior samples (visual)	Whether sampling from N(0,1) produces plausible outputs

Chapter 24: Exercises and Practice Problems

These problems are designed to be solved with pen and paper first, then checked against the solutions below. They cover Parts II and III, which is where most of the conceptual weight of this book lives.

24.1 Conceptual Questions

1. In your own words, explain why a plain Autoencoder's latent space usually contains gaps and holes.
2. Why can't we compute the true posterior $p(z|x)$ directly? Name the specific mathematical object that makes it intractable.
3. What does it mean for the ELBO to be a "lower bound" on $\log p(x)$? Why is maximizing a lower bound a reasonable thing to do?
4. Explain, without equations, what the reparameterization trick accomplishes and why it is necessary.
5. A colleague sets $\beta = 50$ in a β-VAE and complains that reconstructions look blurry and generic. Explain why, in terms of the loss function.
6. Why does VQ-VAE not need a KL-divergence term in its loss function at all?

24.2 Computation Practice

1. Given $\mu = [1.0]$ and $\log\sigma^2 = [0.0]$ for a single latent dimension, compute $D_{KL}(N(\mu,\sigma^2) \,||\, N(0,1))$ by hand using the closed-form formula from Chapter 6.
2. Using the same μ and $\log\sigma^2$ as above, and $\varepsilon = -0.5$, compute the sampled value z using the reparameterization trick.
3. A VAE encoder outputs $\mu = [0, 0, 0]$ and $\log\sigma^2 = [0, 0, 0]$ for every single input in a dataset. What does this tell you about the state of training (refer to Chapter 12)?
4. For a Gaussian with $\sigma^2 = 4$, what is $\log\sigma^2$? (This kind of quick mental conversion is useful when reading VAE code.)

24.3 Design Questions

1. You are building a VAE to compress LiDAR point-cloud frames for a mobile robot's world model. What architectural changes would you consider compared to the image-based ConvVAE in Chapter 14?
2. You need a generative model that produces very sharp, photorealistic textures for a simulation environment, and training time is not a constraint. Would you reach for a vanilla VAE, a GAN, or a Diffusion Model? Justify your choice using Chapter 16's comparison table.
3. You want to generate a specific class of object (e.g. only "red cubes") on demand from a trained generative model. Which VAE variant from Part IV directly solves this, and how?

24.4 Solutions

Conceptual Questions

- **Q1** — The loss function only requires each training point to map to *some* code that decodes back to it. Nothing constrains the space between encoded points, so the encoder is free to leave large unused regions, as long as reconstruction loss on the actual training data is low.
- **Q2** — The evidence term $p(x) = \int p(x|z)p(z)dz$, an integral over every possible z, has no closed-form solution in general, and the true posterior's definition depends directly on this same intractable p(x).
- **Q3** — The ELBO sits below log p(x) with a gap equal to $D_{KL}(q(z|x)||p(z|x))$, which is always ≥ 0. Pushing the ELBO up therefore guarantees log p(x) is at least that high too — we are raising a guaranteed floor, which is the best we can do without being able to compute the true quantity directly.
- **Q4** — It rewrites random sampling $z \sim N(\mu,\sigma^2)$ as a deterministic function $z=\mu+\sigma\cdot\varepsilon$ of a fixed, parameter-free noise source $\varepsilon \sim N(0,1)$. This is necessary because standard backpropagation requires a differentiable path from the loss back to μ and σ, and a raw sampling operation has no such differentiable path.
- **Q5** — A large β multiplies the KL term, so the optimizer is pushed very hard to make every q(z|x) look like the plain prior N(0,1), regardless of the reconstruction cost. The model trades away reconstruction accuracy for a more strongly regularized (and more disentangled) latent space.
- **Q6** — VQ-VAE's latent space is discrete (a finite codebook), not a continuous Gaussian, so there is no meaningful Gaussian KL divergence to compute against a continuous prior in the first place; regularization instead comes from the codebook commitment and codebook-update losses.

Computation Practice

- **Q1** — $D_{KL} = ½\times(1.0^2 + e^0 - 0 - 1) = ½\times(1 + 1 - 0 - 1) = ½\times1 =$ **0.5**
- **Q2** — $\sigma = \exp(0.5\times0) = 1.0$. $z = \mu + \sigma\cdot\varepsilon = 1.0 + (1.0 \times -0.5) =$ **0.5**
- **Q3** — This means μ and σ exactly match the prior N(0,1) for every input — the classic signature of **posterior collapse** (Chapter 12.3): the encoder has given up encoding any information about individual inputs.
- **Q4** — $\log(4) \approx$ **1.386**

Design Questions (Sample Reasoning)

- **Q1** — Point clouds are unordered and irregular, unlike a fixed pixel grid, so plain 2D convolutions are a poor fit. Options include voxelizing the point cloud first (then using 3D convolutions), or using point-based architectures such as PointNet-style layers as the encoder, keeping the VAE loss framework from Chapter 9 unchanged.

- **Q2** — Diffusion Models, per the comparison table in Chapter 16: they produce the sharpest, most realistic textures and are far more stable to train than GANs, at the acceptable cost (since training time isn't constrained) of slower iterative sampling.
- **Q3** — The Conditional VAE (Chapter 15): feed the class label y into both encoder and decoder, then at generation time fix y = “red cube” and sample z randomly to get varied examples of that specific class.

Appendix A: Frequently Asked Questions

Is z supposed to be a single vector, or a distribution?

Both, at different moments. $q_\phi(z|x)$ is a distribution (defined by μ and σ). A specific z is a single vector sampled from that distribution via the reparameterization trick. When people say "the VAE's latent representation of x" loosely, they usually mean μ — the center of that distribution — used as a deterministic point estimate for tasks like interpolation (Chapter 18).

Why is the prior always N(0,1) and not something else?

It is simple, easy to sample from, has a convenient closed-form KL divergence against any Gaussian (Chapter 6), and being centered at the origin with unit variance gives a natural, bounded region to sample new latent codes from at generation time. Other priors (e.g. mixtures of Gaussians, or the discrete codebook in VQ-VAE) are used in specialized variants, but N(0,1) is the default for good reason.

Does a bigger latent dimension always mean a better VAE?

No. Too small, and the model cannot capture enough variation to reconstruct well. Too large, and the model has more room to leave parts of latent space poorly regularized or redundant, and training can become less stable. Latent dimension is a genuine hyperparameter to tune, not a "bigger is always better" setting.

Why do VAE samples often look blurrier than GAN samples?

Partly because pixel-wise reconstruction losses (MSE, BCE) tend to reward "safe," averaged-looking outputs when the model is uncertain, rather than committing to one sharp, specific answer. GANs avoid this because their discriminator directly penalizes anything that looks blurry or unrealistic, rather than rewarding closeness to an average of plausible outputs.

Is the ELBO ever equal to log p(x) exactly?

Yes, exactly when the KL gap term $D_{KL}(q_\phi(z|x) \,||\, p_\theta(z|x))$ is exactly zero — in other words, when the encoder's approximate posterior perfectly matches the true posterior. In practice this is never achieved exactly, but a well-trained encoder gets close.

Appendix B: Notation Reference

Symbol	Meaning
x	An observed data point (e.g. an image)
$\hat{x}$	The reconstruction of x produced by the decoder
z	A latent code / latent vector
θ	Decoder (generative network) parameters
ϕ	Encoder (inference network) parameters
p(z)	The prior distribution over latent codes, usually N(0,1)
$p_\theta(x\|z)$	The decoder / likelihood: probability of x given a latent code z
$q_\phi(z\|x)$	The encoder / approximate posterior: probability of z given input x
$p_\theta(z\|x)$	The TRUE posterior — intractable, approximated by $q_\phi(z\|x)$
μ, σ	Mean and standard deviation output by the encoder
ε	Standard normal noise used in the reparameterization trick
$D_{KL}(\cdot \,\|\|\, \cdot)$	Kullback-Leibler divergence between two distributions
ELBO	Evidence Lower BOund — the quantity we actually maximize
β	Extra weight on the KL term in β-VAE

Appendix C: Glossary

- **Autoencoder** — a neural network trained to reconstruct its own input through a compressive bottleneck.
- **Latent space** — the low-dimensional space in which compressed representations (latent codes) live.
- **Bottleneck** — the smallest, most compressed layer in an encoder-decoder network.
- **Generative model** — a model that learns to produce new samples resembling its training data.
- **Self-supervised learning** — training with labels manufactured automatically from the input data itself, without external annotation.
- **Prior** — an assumed distribution over latent variables, chosen before seeing data (e.g. N(0,1)).
- **Posterior** — the distribution over latent variables after accounting for an observed data point.
- **Variational inference** — approximating an intractable probability distribution with the closest member of a simpler, tractable family.
- **KL divergence** — a (non-symmetric) measure of how different one probability distribution is from another.
- **Evidence Lower Bound (ELBO)** — a tractable lower bound on the log-likelihood of data, maximized in place of the true (intractable) likelihood.
- **Reparameterization trick** — rewriting a random sampling step as a deterministic function of a fixed-distribution noise variable, to allow gradients to flow through it.
- **Posterior collapse** — a VAE failure mode where the decoder ignores the latent code entirely.
- **Disentanglement** — the property of a latent space where each dimension independently controls one interpretable factor of variation.
- **Codebook (VQ-VAE)** — a finite set of learned vectors used to discretize a continuous latent space.
- **World Model** — a learned, compact latent representation of an environment used for planning or control in robotics/RL.

Appendix D: Further Reading

- Kingma, D. P., & Welling, M. (2013). Auto-Encoding Variational Bayes. — the original VAE paper; the source of the derivation in Chapter 9.
- Rezende, D. J., Mohamed, S., & Wierstra, D. (2014). Stochastic Backpropagation and Approximate Inference in Deep Generative Models. — an independent, parallel derivation of the reparameterization trick.
- Higgins, I., et al. (2017). β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework.
- van den Oord, A., Vinyals, O., & Kavukcuoglu, K. (2017). Neural Discrete Representation Learning (VQ-VAE).
- Goodfellow, I., et al. (2014). Generative Adversarial Networks.
- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models.
- Rombach, R., et al. (2022). High-Resolution Image Synthesis with Latent Diffusion Models (Stable Diffusion).
- Ha, D., & Schmidhuber, J. (2018). World Models.
- Doersch, C. (2016). Tutorial on Variational Autoencoders. — an accessible, in-depth companion to the original paper.

Appendix E: Full Formula Cheat Sheet

Name	Formula
AE reconstruction loss (MSE)	$L = \|\|x - \hat{x}\|\|^2$
Gaussian KL to standard normal	$D_{KL} = \frac{1}{2} \cdot \Sigma(\mu^2 + \sigma^2 - \log\sigma^2 - 1)$
Reparameterization trick	$z = \mu + \sigma \cdot \varepsilon,\ \varepsilon \sim N(0,1)$
Standard deviation from log-variance	$\sigma = \exp(0.5 \cdot \log\sigma^2)$
ELBO (general form)	$ELBO(x) = E_q[\log p_\theta(x,z) - \log q_\phi(z\|x)]$
ELBO (practical two-term form)	$ELBO(x) = E_q[\log p_\theta(x\|z)] - D_{KL}(q_\phi(z\|x)\|\|p(z))$
VAE training loss	$L = Reconstruction_Loss + D_{KL}$
β-VAE loss	$L = Reconstruction_Loss + \beta \cdot D_{KL}$
Bayes' rule	$p(z\|x) = p(x\|z) \cdot p(z) / p(x)$
Evidence (intractable)	$p(x) = \int p(x\|z) \cdot p(z)\, dz$
IWAE bound (K samples)	$L_K = E[\log((1/K)\Sigma_i\ p_\theta(x,z_i)/q_\phi(z_i\|x))]$

Appendix F: Index of Figures

Figure	Topic
2.1	Plain Autoencoder architecture
4.1	Plain AE vs. VAE latent space shape
6.1	KL divergence between two Gaussians
7.1	The probabilistic graphical model of a VAE
9.1	ELBO decomposition into reconstruction + KL
10.1	The reparameterization trick
11.1	Full VAE architecture forward pass
12.1	Typical VAE training curves
12.2	Posterior collapse: healthy vs. collapsed KL
13.1	Conditional VAE (CVAE) architecture
15.1	VQ-VAE codebook quantization
16.1	Generative modeling family tree
18.1	Latent space interpolation

Appendix G: Debugging Checklist for VAE Code

A practical, symptom-first checklist for when your own VAE implementation isn't training the way this book describes.

Symptom: Loss is NaN almost immediately

- Check you are exponentiating **$\log\sigma^2$** correctly ($\sigma = \exp(0.5 \cdot \log\sigma^2)$), not accidentally treating the raw encoder output as σ^2 or σ directly.
- Clip or initialize $\log\sigma^2$ carefully — extremely large positive values will explode $\sigma^2 = \exp(\log\sigma^2)$ immediately.
- Verify the reconstruction loss's input range matches its formula (e.g. Sigmoid outputs in [0,1] paired with Binary Cross-Entropy, not MSE against unbounded values).

Symptom: Reconstructions are blurry and generic, all samples look alike

- Check for posterior collapse (Chapter 12.3) — print the average KL term per batch; if it is near zero throughout training, this is very likely the cause.
- Try KL annealing (Chapter 12.4) instead of applying the full KL weight from step one.
- Confirm you are not accidentally using an overly powerful decoder that can reconstruct plausible-looking outputs while ignoring z almost entirely.

Symptom: Random samples from the prior look like garbage

- Confirm the KL term is actually included in the loss and its gradient is flowing — a common bug is computing it but forgetting to add it into the total loss that gets `.backward()`-ed.
- Check that new samples are drawn from exactly the same prior used during training, N(0,1), with the correct number of latent dimensions.
- Verify the model was trained long enough for the KL term to meaningfully regularize the space (see Figure 12.1) — an under-trained VAE behaves like a plain Autoencoder with slight noise.

Symptom: Training loss goes down but generated samples don't improve

- Make sure you are inspecting **freshly sampled** outputs (decoder fed random z from the prior), not reconstructions of real inputs — these can diverge in quality, especially early in training.
- Double check the reconstruction loss uses `reduction="sum"` (or is scaled consistently) to match the derivation in Chapter 9 — mixing a mean-reduced reconstruction loss with a sum-reduced KL term silently changes their relative weighting.

End of textbook.